Welcome Offer Access to everything. Now 30% off. Upgrade now

Data Engineering | Big Data | Data Engineer | Databricks | Python Programming

The Last Comparison You'll Ever Need: Delta Lake vs Apache Iceberg vs Apache Hudi — From First Principles to Production Decisions



Anurag Sharma 15 min read · 3 days ago



If you're an architect, engineer, or data practitioner who has ever wondered, "Which table format should I pick — and what happens if I choose wrong?" — this blog was written for you.

Before we dive in, ask yourself honestly: is even one of these true for you?

- ✅ You want a deep, architectural understanding of how these formats actually work under the hood — not just feature bullet points.
- ✅ You want a clear, honest view of the pros and cons of each format so you can make a defensible decision.
- ✅ As an architect, you need to know when to switch formats, and if you do, what a real migration plan looks like.

- ✅ You want to know what comes after these formats — where the industry is heading in 2026 and beyond.

If any one of those is **YES**, keep reading. This is the only comparison you'll need.

What We Cover

1. The "why" — what problem all three formats solve
2. How each format works, from high level to deepest internals
3. Architecture diagrams explained
4. Side-by-side feature comparison
5. When to choose each one — and when the "right" choice becomes the wrong one
6. Format conversions: every direction, every tool
7. Migration strategy and decision framework
8. Testing Your Conversion
9. What's next — the convergence era

Part 1 — The Problem All Three Formats Solve

Object storage (S3, GCS, ADLS) is cheap, durable, and infinitely scalable. But raw files in a bucket have no concept of transactions, schema, history, or updates. You end up with:

- Broken pipelines when two jobs write simultaneously
- No way to "undo" a bad load

- Partition rewrites to simulate an UPDATE
- Schema mismatch errors that only surface at query time
- Full table scans because there's no metadata to prune files

Delta Lake, Apache Iceberg, and Apache Hudi all solve the same fundamental problem: they add a *metadata layer* on top of Parquet/ORC files that provides database-grade semantics — ACID transactions, schema evolution, time travel, row-level updates and deletes — without requiring a traditional database.

> *Think of them as an "operating system" sitting between your compute engines and your raw object storage.*

All three deliver these core capabilities:

Capability	All Three
ACID Transactions	✅
Snapshot Isolation	✅
Schema Evolution	✅
Time Travel	✅
Row-Level Updates/Deletes	✅
Partition Pruning	✅

The differences are in *philosophy, architecture, and the tradeoffs that follow from both.*

Part 2 — Origins and Philosophy

Understanding *who built these and why* is the fastest way to understand their design decisions.

Attribute	Apache Iceberg	Delta Lake	Apache Hudi
Created by	Netflix	Databricks	Uber
Year (OSS)	2018	2019	2017
Governance	Apache Software Foundation	Linux Foundation	Apache Software Foundation
Latest Version (2026)	1.11.0	4.3.0	1.2.0
Core Design Philosophy	Vendor-neutral specification first	Spark-native, broaden later	Upsert-first, streaming-native

Netflix built **Iceberg** because they were drowning in Hive table management complexity at billion-file scale. Their goal: make a format that any engine could read correctly — a *spec before an implementation*.

Databricks built **Delta** because Spark needed reliable, production-grade writes. Their goal: make Spark pipelines reliable and fast, with full integration into their platform.

Uber built **Hudi** because they had one specific hard problem: ingest millions of CDC (Change Data Capture) events per second into a data lake with record-level upserts, and make those updates queryable near-real-time. Everything else in Hudi's design flows from that.

Part 3 — How Each Format Works: Deep Architecture

🔷 Apache Iceberg — The Specification-First Format

High-Level View: Iceberg is a *table specification*. It defines a metadata hierarchy that any engine can implement independently. It doesn't care what engine writes or reads — Spark, Flink, Trino, DuckDB, BigQuery, Snowflake: all are first-class citizens.

The Metadata Hierarchy (Deepest Level):

```
Catalog (REST / Glue / Hive Metastore / Nessie)
    └── Metadata Pointer (points to current metadata.json)
            └── Metadata File (metadata.json)
                    ├── Schema
                    ├── Partition Spec
                    ├── Sort Order
                    └── Snapshot List
                            └── Manifest List (snap-XXXX.avro)
                                    └── Manifest Files (*.avro)
                                            └── Data Files (*.parquet / *.orc)
                                                    └── Column-level statistics
                                                            (min/max/null counts per file)
```

How a Write Works:

1. New data files are written.
2. A new manifest file is created, listing those files plus their column statistics.
3. A new manifest list (snapshot) is created, referencing all manifest files.
4. A new metadata.json is written, referencing the new snapshot.
5. The catalog atomically swaps the pointer to the new metadata.json.

The Power of This Design:

- File pruning happens by reading compact manifests — not by listing millions of S3 objects.
- *Hidden partitioning*: partition columns are not exposed to queries. You write, `WHERE event_date = '2026-01-01'` and Iceberg figures out which files to read. Changing partition strategy never breaks existing queries.
- *Partition evolution*: you can change how a table is partitioned in-place. Old data stays under the old scheme, new data lands under the new scheme — zero rewrite.
- Iceberg v3 (GA in 2026) adds: deletion vectors, row lineage, VARIANT type for semi-structured data, nanosecond timestamps, and native geospatial types.

🔶 Delta Lake — The Transaction Log Format

High-Level View: Delta Lake is deeply tied to its `_delta_log` — a directory of ordered JSON commit files that record every change ever made to a table. It grew out of Databricks' need to make Spark pipelines reliable, and that heritage shows in its simplicity and tight Spark integration.

The Metadata Structure (Deepest Level):

```
Table Root/
    ├── _delta_log/
    │       ├── 00000000000000000000.json  ← commit 0: table creation
    │       ├── 00000000000000000001.json  ← commit 1: data added
    │       ├── 00000000000000000002.json  ← commit 2: schema change
```

```
│       ├── ...
│       └── 00000000000000000010.checkpoint.parquet  ← compacted log (every
└── part-00001.snappy.parquet
└── part-00002.snappy.parquet
```

How a Write Works:

1. New Parquet files are written.

2. A new JSON commit entry is appended to `_delta_log`, listing:

- Files added (`add` actions)
- Files removed (`remove` actions)
- Metadata changes (schema, partition spec)

3. Every N commits (default 10), the log is compacted into a Parquet checkpoint for fast state reconstruction.

Key Delta-Specific Features:

- Liquid Clustering (Delta 3.0+): replaces traditional static partitioning with a dynamic clustering strategy that automatically reorganizes data. No

Open in app ↗

Medium Search Write

- Deletion Vectors (Delta 2.3+): instead of rewriting files on DELETE/UPDATE, Delta writes a small bitmap "deletion vector" file marking which rows are deleted. Massive speedup for row-level operations.

- Coordinated Commits (Delta 4.0): moves commit coordination to the catalog rather than S3 object listing, dramatically improving concurrent write performance.
- Delta UniForm: writes Delta metadata while *simultaneously generating Iceberg (and Hudi) metadata* over the same files — no data copy. A single physical table readable by all three formats.

🟠 Apache Hudi — The Database-Grade Ingestion Engine

High-Level View: Hudi is not just a table format — it is a *Data Lakehouse Management System (DLMS)*. It has a record-level index, a full timeline of every operation (like a database WAL), two storage layouts (CoW and MoR), and built-in table services that run automatically to maintain health.

The Architecture (Deepest Level):

```
Table Root/
    ├── .hoodie/
    │       ├── .hoodie_properties        ← table config: type, key, schema
    │       ├── 20260101120000.commit      ← timeline actions
    │       ├── 20260101120000.deltacommit
    │       ├── 20260101120000.compaction
    │       ├── 20260101120000.clean
    │       └── .index/                   ← record-level index (HFiles, Bloom)
    ├── partition=2026-01-01/
    │       ├── base_file.parquet         ← CoW: full columnar data
    │       └── delta_log.avro            ← MoR: incremental row changes
```

Two Storage Layouts — This is Hudi's Superpower:

Layout	How it Works	Best For
Copy-on-Write (CoW)	On every UPDATE/DELETE, affected Parquet files are fully rewritten	Read-heavy, low update frequency
Merge-on-Read (MoR)	Updates written to row-based Avro delta logs; merged at read time or via async compaction	High-frequency upserts, streaming ingestion

Hudi's Record-Level Index: This is what makes Hudi fundamentally different. When you do an upsert, Hudi doesn't scan the whole table — it uses an index to find *exactly which file* contains that record key and updates only that file. This enables true database-grade upsert performance at lake scale.

Index types:

- Bloom Filter Index (default): probabilistic, fast, good for medium cardinality
- HBase Index: external, strongest for random lookups at massive scale
- In-Memory Index: fastest for small tables
- Bucket Index: deterministic file placement by hash
- LSM-tree-based Timeline (Hudi 1.0+): replaces the flat `.hoodie` timeline for tables with billions of records and long retention windows

Hudi's Table Services (autonomous, run in background):

- Compaction: merges MoR delta logs into base Parquet files
- Clustering: reorganizes files for better query performance
- Cleaning: removes old file versions beyond retention
- Indexing: builds and maintains record-level indexes

Part 4 — The Definitive Feature Comparison

Capability	Apache Iceberg	Delta Lake	Apache Hudi
Read-heavy analytics	⭐⭐⭐ Strong	⭐⭐⭐ Strong	⭐⭐ Good
Streaming upserts / CDC	⭐⭐ Good	⭐⭐ Good	⭐⭐⭐ Strong
Multi-engine portability	⭐⭐⭐ Strong	⭐⭐ Good	⭐⭐ Good
Partition evolution	⭐⭐⭐ Strong	⭐ Limited (clustering only)	⭐ Limited
Hidden partitioning	⭐⭐⭐ Yes	❌ No	❌ No
Record-level indexing	⭐⭐ Good	⭐⭐ Good	⭐⭐⭐ Strong
Concurrent writers	⭐⭐ Good (OCC)	⭐⭐⭐ Strong (Coordinated)	⭐⭐⭐ Strong (NBCC)
Cloud vendor support	⭐⭐⭐ Strong	⭐⭐ Good	⭐ Limited
Catalog ecosystem	⭐⭐⭐ Strong	⭐⭐ Good	⭐ Limited
Streaming ingestion	⭐⭐ Good	⭐⭐ Good	⭐⭐⭐ Strong (native)
Semi-structured (VARIANT)	⭐⭐⭐ v3	⭐⭐⭐ 4.0	⭐⭐ Good
Snowflake support	⭐⭐⭐ Native	⭐⭐ Via UniForm	❌ No
Microsoft Fabric	⭐⭐ Via UniForm	⭐⭐⭐ Native	❌ No
Databricks	⭐⭐⭐ Native	⭐⭐⭐ Native	⭐⭐ Supported

Part 5 — The Architect's Decision Framework: When to Choose What

Choose Apache Iceberg When:

- You are building a new, vendor-neutral lakehouse and don't want to be locked into any single platform.

- You have multi-engine access patterns — Spark for ETL, Trino/Athena for ad-hoc queries, Snowflake for BI, Flink for streaming. Iceberg is a first-class citizen everywhere.
- Your primary workload is batch ETL and analytics with occasional updates.
- You need partition evolution without table rewrites. This is a real operational superpower for long-lived tables.
- You're on AWS (S3 Tables, Athena, Glue) or Snowflake — both default to Iceberg.
- You care about long-term format durability — Iceberg v3 is the industry convergence standard, and even Databricks is proposing Delta 5.0 share Iceberg's metadata tree.

When Iceberg is the Wrong Choice:

- You need extremely high-frequency record-level upserts with low latency (thousands of updates/second). Iceberg's metadata overhead per commit becomes a bottleneck without external solutions.
- You are all-in on Databricks and don't need engine portability — Delta is the obvious choice there.
- You need native streaming CDC ingestion without building your own infrastructure.

Choose Delta Lake When:

- You are deeply invested in the Databricks ecosystem or Microsoft Fabric/Azure — Delta is native and deeply optimized there.
- Your team thinks in Spark and values simplicity. Delta's transaction log is dead simple to reason about.

- You need Liquid Clustering — Delta's dynamic clustering is genuinely superior to managing static partition columns on frequently queried, high-cardinality tables.
- You want a battle-tested format with the largest community of Databricks engineers behind it.
- You use Delta UniForm to cover multi-engine needs without changing your write path.

When Delta is the Wrong Choice:

- You need true engine-agnostic portability today — while UniForm helps, Iceberg is still the cleaner multi-engine story.
- You are on a non-Databricks stack (pure Snowflake, Athena, BigQuery) — Delta integration requires workarounds.
- You need advanced partition evolution — Delta doesn't support it. You need Liquid Clustering, which requires compute to backfill.

Choose Apache Hudi When:

- Your primary problem is CDC / streaming upserts — this is what Hudi was born to do, and it does it better than the others.
- You run near-real-time pipelines where MoR allows you to make updates queryable within seconds, with compaction catching up asynchronously.
- You need record-level change streams as a first-class output — Hudi's incremental query mode lets downstream consumers process only changes, not full table scans.
- Your workload is mutable by nature — event corrections, late arrivals, GDPR deletes at scale.

- You are comfortable operating the full Hudi platform (timeline, compaction, cleaning, indexing) — there is more operational surface area, but it pays off for the right workload.

When Hudi is the Wrong Choice:

- You need broad platform support — Hudi has limited native support on Snowflake, Microsoft Fabric, and several managed query services. This is the biggest Hudi drawback in 2026.
- Your workload is append-only analytics — you're paying Hudi's write overhead for a benefit you don't need.
- Your team doesn't have the operational maturity to manage Hudi's table services.

Part 6 — Format Conversions: Every Direction, Every Tool

This is the section that most comparison blogs skip. You picked a format three years ago, your needs changed, and now you need out. Here's how to do it safely.

The Two Tools That Changed Everything

🔁 Apache XTable (formerly OneTable, now Apache-incubating)

XTable performs omnidirectional metadata translation between all three formats. It reads the source format's metadata and generates the target format's metadata *pointing at the exact same Parquet files*. No data is copied or rewritten.

```
# XTable config: translate a Hudi table to Iceberg
sourceFormat: HUDI
targetFormats:
  - ICEBERG
datasets:
  - tableBasePath: s3://warehouse/orders
    tableName: orders
    partitionSpec: order_date:VALUE
```

```
java -jar xtable-utilities-bundled.jar --datasetConfig my_config.yaml
```

XTable syncs incrementally — on re-run, it only translates new commits.

🔶 Delta UniForm

UniForm is built into Delta Lake itself. When enabled on a Delta table, every Delta commit simultaneously generates Iceberg (and optionally Hudi) metadata over the same files. No separate job is needed.

```
-- Enable UniForm on an existing Delta table
ALTER TABLE my_table SET TBLPROPERTIES (
  'delta.universalFormat.enabledFormats' = 'iceberg'
);
```

All 6 Conversion Paths

1️⃣ Delta → Iceberg

Option A — Metadata translation (no data copy, read-only Iceberg view): Use Delta UniForm. The Iceberg view is read-only; writes must still go through Delta.



Option B — Metadata adoption (no data copy, full Iceberg table): Use Apache XTable or Dremio CTAS:

```
-- Dremio: convert Delta to a full Iceberg table
CREATE TABLE iceberg_catalog.db.orders AS
SELECT * FROM delta_source.db.orders;
```

Option C — Full rewrite (clean start): Use Spark CTAS to write a brand-new Iceberg table:

```
spark.read.format("delta").load("s3://warehouse/orders") \
    .write.format("iceberg").save("s3://iceberg/orders")
```

Best when you also want to re-optimize partitioning and file sizes.

Decision Guide: Use UniForm if you just need Iceberg read access. Use XTable or CTAS if you're permanently switching write paths.

2️⃣ Iceberg → Delta

Option A — XTable metadata translation:

```
sourceFormat: ICEBERG
targetFormats:
  - DELTA
datasets:
  - tableBasePath: s3://warehouse/orders
```

Option B — Full rewrite (CTAS):

```
spark.read.format("iceberg").load("iceberg.db.orders") \
    .write.format("delta").save("s3://delta/orders")
```

> ***Watch Out:*** *Delta doesn't support Iceberg's hidden partitioning. Map partition columns explicitly when rewriting.*

3️⃣ Delta → Hudi

Use XTable:

```
sourceFormat: DELTA
targetFormats:
  - HUDI
datasets:
  - tableBasePath: s3://warehouse/orders
    tableName: orders
```

Or a full CTAS rewrite with the Hudi writer configured for the record key and partition fields.

4️⃣ Hudi → Delta

Use XTable:

```
sourceFormat: HUDI
targetFormats:
  - DELTA
```

5️⃣ Iceberg → Hudi

Use XTable (the `tableDataPath` is required for Iceberg when data files live in a subdirectory):

```
sourceFormat: ICEBERG
targetFormats:
  - HUDI
datasets:
  - tableBasePath: s3://warehouse/orders
    tableDataPath: s3://warehouse/orders/data
    tableName: orders
    partitionSpec: order_date:VALUE
```

The XTable run produces Hudi metadata under the table's base path. Re-run on a schedule for ongoing sync.

Full migration sequence (writer cutover):

1. Stop the Iceberg writer at a clean commit boundary.
2. Run a final XTable sync (Iceberg → Hudi).
3. Start the Hudi writer against the same base path.

4. Enable reverse XTable sync (Hudi → Iceberg) so existing Iceberg readers keep working.
5. Migrate readers to native Hudi on their own schedule.

6️⃣ Hudi → Iceberg

The same XTable two-way pattern in reverse:

```
sourceFormat: HUDI
targetFormats:
  - ICEBERG
```

Enable continuous XTable sync post-cutover so Hudi readers can be migrated incrementally.

> ⚠️ Always validate row counts and checksums before cutting over writers. Run source and target in parallel for at least one full ETL cycle before decommissioning the source.

Part 7 — Migration Decision Framework for Architects

When you're evaluating whether to migrate, the answer is rarely "yes, migrate immediately." Use this framework:

```
START
  |
  ▼
Is your current format causing an active production problem?
  | YES → Is it a write performance issue?
  |              YES → Consider Hudi (MoR + indexing)
  |              NO  → Is it engine portability?
  |                          YES → Migrate to Iceberg
  |                          NO  → Is it platform lock-in?
  |                                    YES → Migrate to Iceberg
  |
  | NO → Are you planning a major re-architecture?
             YES → Choose Iceberg (safe default, widest support)
             NO  → Stay where you are; use XTable/UniForm for interoperability
```

The Golden Rule of Format Migration: The format is rarely the bottleneck. Before you migrate, verify the problem isn't in your pipeline logic, cluster sizing, or file compaction strategy.

Part 8— Testing Your Conversion: A Practical Validator's Guide

Before cutting over any migration, you must run thorough validation tests. Skipping this is how production pipelines break at 3 AM. Here's a replicable testing workflow.

Step 1: Row-Count Validation (Basic, But Essential)

Run identical queries in both source and target formats and compare counts:

```
# Delta → Iceberg validation example
delta_count = spark.read.format("delta").load("s3://warehouse/orders").count()
iceberg_count = spark.read.format("iceberg").load("iceberg_db.orders").count()

assert delta_count == iceberg_count, f"Mismatch: Delta={delta_count}, Iceberg={i

# Bonus: Validate by partition when reasonable
from pyspark.sql.functions import col
partition_col = "order_date"

delta_df = spark.read.format("delta").load("s3://warehouse/orders")
iceberg_df = spark.read.format("iceberg").load("iceberg_db.orders")

delta_by_partition = delta_df.groupBy(col(partition_col)).count().collect()
iceberg_by_partition = iceberg_df.groupBy(col(partition_col)).count().collect()

# Convert to dicts for comparison
delta_dict = {row[partition_col]: row["count"] for row in delta_by_partition}
iceberg_dict = {row[partition_col]: row["count"] for row in iceberg_by_partition

for key in set(delta_dict.keys()) | set(iceberg_dict.keys()):
    if delta_dict.get(key, 0) != iceberg_dict.get(key, 0):
        print(f"Partition {key}: Delta={delta_dict.get(key,0)}, Iceberg={iceberg
```

Step 2: Checksum Validation (Guards Against Data Corruption)

Compare cryptographic hashes of your data:

```
from pyspark.sql.functions import sha2, concat_ws, col
import json

# Create deterministic ordering by primary/partition keys
order_cols = ["order_id", "order_date", "customer_id"]  # adjust to your schema

delta_df = spark.read.format("delta").load("s3://warehouse/orders")
iceberg_df = spark.read.format("iceberg").load("iceberg_db.orders")

# Generate row-level hash
for col_name in delta_df.columns:
    delta_df = delta_df.withColumn(col_name, col(col_name).cast("string"))

delta_hash_df = delta_df.withColumn(
```

```
    "row_hash",
    sha2(concat_ws("|", *[col(c) for c in delta_df.columns]), 256)
)

iceberg_hash_df = iceberg_df.withColumn(
    "row_hash",
    sha2(concat_ws("|", *[col(c) for c in iceberg_df.columns]), 256)
)

# Compare aggregated hash
delta_final_hash = delta_hash_df.agg({"row_hash": "min"}).collect()[0][0]
iceberg_final_hash = iceberg_hash_df.agg({"row_hash": "min"}).collect()[0][0]

print(f"Delta aggregated hash: {delta_final_hash}")
print(f"Iceberg aggregated hash: {iceberg_final_hash}")
```

Step 3: Sample Drill-Down (Catch Data Type Issues)

Randomly sample and compare specific records:

```
# Sample 100 random records
sample_fraction = 100.0 / delta_count if delta_count > 100 else 1.0

delta_sample = delta_df.sample(fraction=sample_fraction, seed=42).cache()
iceberg_sample = iceberg_df.sample(fraction=sample_fraction, seed=42).cache()

# Convert both to list of dicts for comparison
delta_list = [row.asDict() for row in delta_sample.collect()]
iceberg_list = [row.asDict() for row in iceberg_sample.collect()]

# Sort by primary key for 1:1 comparison
primary_key = "order_id"
delta_list.sort(key=lambda x: x[primary_key])
iceberg_list.sort(key=lambda x: x[primary_key])

for i, (d_row, i_row) in enumerate(zip(delta_list, iceberg_list)):
    if d_row != i_row:
        print(f"Mismatch at sample index {i}:")
        print(f"  Delta:   {json.dumps(d_row, default=str)}")
        print(f"  Iceberg: {json.dumps(i_row, default=str)}")
        # Check field-by-field
        for key in d_row.keys():
```

```
            if d_row.get(key) != i_row.get(key):
                print(f"    Field '{key}': Delta='{d_row.get(key)}', Iceberg='{i
```

Step 4: Schema Compatibility Test

Ensure schema evolution works as expected:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# Get schemas
delta_schema = spark.read.format("delta").load("s3://warehouse/orders").schema
iceberg_schema = spark.read.format("iceberg").load("iceberg_db.orders").schema

print("Schema Comparison:")
print("Delta Schema:", delta_schema.treeString())
print("Iceberg Schema:", iceberg_schema.treeString())

# Check nullable compatibility
for delta_field, iceberg_field in zip(delta_schema.fields, iceberg_schema.fields
    if delta_field.name != iceberg_field.name:
        print(f"Field name mismatch: Delta '{delta_field.name}' vs Iceberg '{ice
    if delta_field.nullable != iceberg_field.nullable:
        print(f"Nullability mismatch for '{delta_field.name}': "
              f"Delta={delta_field.nullable}, Iceberg={iceberg_field.nullable}")
```

Step 5: Performance Smoke Test

Run a handful of representative queries on both sides:

```
# Example queries that represent your workload
test_queries = [
    "SELECT COUNT(*) FROM table WHERE order_date = '2024-01-01'",
    "SELECT customer_id, SUM(order_amount) FROM table GROUP BY customer_id HAVIN
    "SELECT * FROM table WHERE order_status = 'SHIPPED' ORDER BY order_date DESC
    "SELECT AVG(order_amount), MIN(order_date), MAX(order_date) FROM table"
]
```

```
for query_template in test_queries:
    delta_query = query_template.replace("table", "delta.`s3://warehouse/orders`
    iceberg_query = query_template.replace("table", "iceberg_db.orders")

    print(f"\nTesting: {query_template}")

    start = time.time()
    delta_result = spark.sql(delta_query).collect()
    delta_time = time.time() - start

    start = time.time()
    iceberg_result = spark.sql(iceberg_query).collect()
    iceberg_time = time.time() - start

    print(f"  Delta time: {delta_time:.2f}s, Iceberg time: {iceberg_time:.2f}s")
    print(f"  Ratio (Iceberg/Delta): {iceberg_time/delta_time:.2f}")
```

Step 6: Parallel Run Validation (The Golden Standard)

The safest migration path: run both formats side-by-side for a full business cycle:

```
# Production pipeline config with dual-write
write_strategy: "dual_write"
source: kafka_topic
sinks:
  - type: delta
    path: s3://warehouse/orders
  - type: iceberg
    path: s3://warehouse/orders-iceberg

validation:
  frequency: hourly
  checks:
    - row_count_match
    - sample_validation: 0.1%
    - partition_integrity: order_date

# Monitor divergence in real-time
metrics_to_alert_on:
  - row_count_difference > 0.1%
```

```
    - any_partition_mismatch
    - schema_compatibility_error
```

Run this dual-write phase for at least one full ETL cycle (daily, weekly) or 7 days before cutting over.

Testing Checklist Before Production Cutover

- Row counts match across all partitions
- Sample validations pass (0 mismatches in 1000+ samples)
- Schema compatibility verified (no data type surprises)
- Performance within 20% variance (for key queries)
- Null handling consistent (NULL vs empty string)
- Partition pruning works in target format
- Time travel queries return identical results
- Concurrent write tests pass (if applicable)
- Backup strategy tested (can rollback if needed)
- Monitoring in place for post-migration

Part 9— What Comes After These Formats? The Convergence Era

This is the most exciting part. The "format war" of 2021–2025 is over. What's happening in 2026 is more interesting: convergence.

Apache Iceberg v4 (In Design)

The most consequential open format work in the industry right now. Planned additions:

- Single-file commits and adaptive metadata tree: makes streaming-rate writes as cheap as Delta's log
- Snapshot offloading and columnar metadata: cuts planning costs for tables with millions of files
- Efficient column updates: targets AI/ML workloads with extremely wide tables

Delta Lake 5.0 (Proposed)

In a move that signals the end of the format war more loudly than any benchmark: Databricks has proposed that Delta Lake 5.0 adopt the Iceberg v4 metadata tree as its native content metadata. One on-disk structure, readable and writable by both Delta and Iceberg clients, with no translation layer. If this lands, the remaining differences between Delta and Iceberg are engineered away at the metadata level itself.

Apache Hudi 1.x Roadmap

Hudi is positioning itself as the database-grade, write-optimized ingestion tier of a multi-format world — the format you use when you need upserts at database speed, with XTable and Iceberg-compatible surfaces handling read-side reach. The roadmap deepens indexing, streaming semantics, and keyed operations.

The New Battleground: Catalogs

The format layer is commoditizing. The next war is at the catalog layer — REST-compatible catalogs (Apache Polaris, Unity Catalog, AWS Glue, Nessie) that provide governance, access control, and cross-engine discoverability. If you're making a format decision today, pair it with a REST-compatible catalog. That choice will matter more in three years than the format itself.

Final Summary — The One Decision

> *Apache Iceberg is the safe, forward-looking default for new lakehouses in 2026 — the broadest engine support, the strongest catalog ecosystem, and every major cloud ships managed Iceberg natively.*
>
> *Delta Lake is the right choice if you're all-in on Databricks or Microsoft Fabric, and its proposed convergence with Iceberg v4 makes it a non-losing bet.*
>
> *Apache Hudi is the specialist choice for CDC-heavy, high-frequency upsert workloads where its record-level indexing and MoR storage genuinely outclass the alternatives.*
>
> *And thanks to Apache XTable and Delta UniForm, this choice is no longer permanent — you can write in one format and read in all three, letting interoperability cover what your primary format doesn't.*

Quick Reference Cheat Sheet

If this saved you a week of research, share it with someone who's currently picking a format.

Data Engineering + Big Data + Data Engineer + Databricks + Python Programming +

Written by Anurag Sharma

136 followers · 8 following

Edit profile

Data Engineering Specialist with 11+ Exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!